

DBMS PROJECT REVIEW -1

TEAM 17

G ANIRUDH 24BDS0335

TANAY MOHTA 24BDS0294

DHYEY DHARMESHBHAI DUMASIA 24BCE0529

1. Problem Definition and Objectives

Problem Definition

Modern microservice architectures often rely on direct, synchronous communication between services. This makes individual service failures propagate through the system. During sudden traffic spikes, a backend service may receive requests faster than it can process them, causing queues to build up, services to become overloaded, and failures to cascade across dependent services.

A conventional Relational Database Management System is designed for structured transactional queries. Using it directly as a high-volume message queue can introduce additional overhead. On the other hand, a high-throughput message broker requires durable storage, tracking, access control, configuration management, and auditing.

Conduit addresses these requirements through a hybrid architecture. It provides a durable message-broker data plane and a RDBMS control and metadata plane for structured administrative, security, configuration, monitoring, and auditing operations.

Objectives

Conduit aims to:

1. Provide asynchronous communication between independent services while decoupling their execution and availability.
2. Provide durable, crash-resilient message persistence using a Write-Ahead Log (WAL), append-only log storage, and recovery mechanisms.
3. Provide an RDBMS-based control and management layer for users, applications, topics, permissions, schemas, configurations, and audit records.
4. Demonstrate core DBMS capabilities through relational constraints, SQL queries, joins, aggregation, indexing, transactions, views, triggers, and procedures/functions.

2.1 Functional Requirements

1. Register and manage system users and client applications.
2. Provide role-based access control and topic-level permissions.
3. Create and manage topics, including partition configuration and schema versions.
4. Persist, consume, and track messages using append-only logs, WAL, offsets, and consumer groups.
5. Provide SQL-based management, monitoring, auditing, and transaction tracking through the administrative dashboard.

2.2 Non-Functional Requirements

1. **Performance:** Ensure fast message processing using sequential log-based storage without synchronous relational operations on the message path.
2. **Durability:** Ensure acknowledged messages remain recoverable after broker crashes through WAL-based recovery.
3. **Scalability:** Support increasing numbers of topics, applications, consumer groups, partitions, and messages.
4. **Security & Integrity:** Protect system data using role-based access control and enforce relational integrity through constraints and transactions.
5. **Concurrency & Availability:** Support concurrent operations and allow message processing to resume from committed offsets after failures.

2.3 Database Design

Conduit follows a hybrid database architecture consisting of a relational control plane and a data plane.

- **RDBMS Control Plane:** The relational database manages structured administrative and operational information such as users, topics, access permissions, consumer groups, audit records, and broker configurations. It provides relational integrity, constraints, SQL-based querying, and transaction management.
- **Data Plane:** This handles the actual high-volume message data using a persistent append-only log, Write-Ahead Logging (WAL), and in-memory indexing for efficient message storage and retrieval.
- **Separation of Responsibilities:** The RDBMS is responsible for structured metadata, administration, access control, and monitoring, while the data plane is responsible for high-throughput message persistence and processing. This prevents administrative database operations from becoming a bottleneck for the message-processing path.
- **Relational Model:** The RDBMS is modelled using six core entities — USER, TOPIC, CONSUMER_GROUP, AUDIT_LOG, BROKER_CONFIG, and ACCESS — with primary keys, foreign keys, and defined relationships between them.
- **Database Integrity and Operations:** The relational layer is designed to support constraints, indexing, joins, aggregation, transactions, and SQL-based reporting required for the administrative dashboard.

3. Database Model / ER Diagram

Entities

1. USER

- user_id — PK
- username
- email
- role
- status

- created_at

2. TOPIC

- topic_id — PK
- topic_name
- description
- created_by — FK → USER
- partition_count
- schema_definition
- retention_period
- status
- created_at

3. CONSUMER_GROUP

- group_id — PK
- group_name
- topic_id — FK → TOPIC
- committed_offset
- status
- created_at

4. AUDIT_LOG

- log_id — PK
- user_id — FK → USER
- topic_id — FK → TOPIC
- event_type
- status
- event_timestamp
- details

5. BROKER_CONFIG

- config_id — PK
- config_key
- config_value
- updated_by — FK → USER
- updated_at

6. ACCESS

- access_id — PK
- user_id — FK → USER
- topic_id — FK → TOPIC
- access_type
- granted_at

Relationships

1. USER 1 : M TOPIC

One user can create multiple topics, while each topic is created by one user.

2. TOPIC 1 : M CONSUMER_GROUP

One topic can have multiple consumer groups, while each consumer group is associated with one topic.

3. **USER 1 : M AUDIT_LOG**

One user can perform multiple auditable actions, while each audit record is associated with a user.

4. **TOPIC 1 : M AUDIT_LOG**

One topic can have multiple audit events, while each topic-related audit record references one topic.

5. **USER 1 : M BROKER_CONFIG**

One user can update multiple broker configuration records, while each configuration record is associated with the user who updated it.

6. **USER 1 : M ACCESS**

One user can have multiple access permissions.

7. **TOPIC 1 : M ACCESS**

One topic can have multiple access permission records.

ER DIAGRAM:

Conduit — Simplified Relational ERD

